

POO en Python L2 MPI & SML

Chapitre 2: Les classes et objets en détail

Pr Amadou Dahirou GUEYE, UAM

Objectifs

Dans ce chapitre, nous allons approfondir la notion de classe et objet, deux concepts fondamentaux en Programmation Orientée Objet (POO). Nous verrons comment définir une classe, instancier des objets, utiliser un constructeur (`__init__`), manipuler les attributs et appeler des méthodes.

Définition et instanciation de classes

Qu'est-ce qu'une classe ?

Une **classe** est un modèle ou un plan qui permet de créer des objets partageant les mêmes caractéristiques et comportements. Elle définit un ensemble d'attributs (variables) et de méthodes (fonctions) qui caractérisent les objets créés à partir de cette classe.

Définition et instanciación de classes

Qu'est-ce qu'un objet ?

Un objet est une instance d'une classe, c'est-à-dire une entité concrète créée à partir du modèle défini par la classe. Chaque objet possède ses propres valeurs pour les attributs définis dans la classe.

Définition et instantiation de classes

Exemple de classe et instantiation d'un objet

Voici un exemple simple d'une classe Voiture et de la création d'un objet à partir de cette classe.

```
# Définition d'une classe Voiture
class Voiture:
    pass # Pour l'instant, la classe est vide

# Instantiation d'un objet
ma_voiture = Voiture()

# Vérification du type de l'objet
print(type(ma_voiture)) # <class '__main__.Voiture'>
```

Définition et instanciación de classes

Explication :

1. La classe ***Voiture*** est définie avec le mot-clé ***class***, mais elle ne contient encore aucun attribut ni méthode.
2. L'objet ***ma_voiture*** est créé à partir de cette classe en appelant `Voiture()`.
3. La fonction ***type(ma_voiture)*** nous montre que l'objet appartient bien à la classe `Voiture`.

Le constructeur `__init__` et le rôle de `self`

Le constructeur `__init__`

Lorsqu'on crée un objet, on veut souvent qu'il ait des valeurs initiales. Le constructeur `__init__` est une méthode spéciale qui est automatiquement appelée lors de l'instanciation d'un objet. Il permet d'initialiser les attributs de l'objet avec des valeurs définies par l'utilisateur.

Le constructeur `__init__` et le rôle de `self`

Le rôle de *self*

Le paramètre `self` fait référence à l'instance courante de la classe et permet d'accéder aux attributs et aux méthodes de l'objet. Il est toujours le premier paramètre des méthodes de classe.

Le constructeur `__init__` et le rôle de `self`

Exemple avec un constructeur `__init__`

```
class Voiture:
    # Constructeur qui initialise les attributs
    def __init__(self, marque, modele):
        self.marque = marque # Attribut d'instance
        self.modele = modele # Attribut d'instance

# Création d'un objet avec des valeurs spécifiques
voiture1 = Voiture("Toyota", "Corolla")

# Affichage des attributs
print(voiture1.marque) # Toyota
print(voiture1.modele) # Corolla
```

Le constructeur `__init__` et le rôle de `self`

Explication :

1. La méthode `__init__` prend deux paramètres (*marque* et *modele*) qui sont affectés aux attributs d'instance (*self.marque* et *self.modele*).
2. Lorsqu'on crée `voiture1`, on passe "Toyota" et "Corolla" en arguments.
3. `voiture1.marque` et `voiture1.modele` stockent respectivement "Toyota" et "Corolla".

Manipulation des attributs et des méthodes

Une méthode

Une méthode est une fonction définie à l'intérieur d'une classe et qui permet d'agir sur les objets de cette classe. Une méthode peut accéder et modifier les attributs d'un objet via self.

Manipulation des attributs et des méthodes

Ajout d'une méthode pour afficher les détails de l'objet

```
class Voiture:
    def __init__(self, marque, modele):
        self.marque = marque
        self.modele = modele

    # Méthode pour afficher les détails de l'objet
    def afficher_details(self):
        print(f"Marque : {self.marque}, Modèle : {self.modele}")

# Création d'un objet et appel d'une méthode
voiture2 = Voiture("Honda", "Civic")
voiture2.afficher_details() # Marque : Honda, Modèle : Civic
```

Manipulation des attributs et des méthodes

Explication :

1. La méthode *afficher_details* affiche les valeurs des attributs de l'objet.
2. *voiture2* est un objet de la classe *Voiture* avec "Honda" et "Civic" comme valeurs.
3. *voiture2.afficher_details()* appelle la méthode, qui affiche :

Marque : Honda, Modèle : Civic

Manipulation des attributs et des méthodes

Modification des attributs d'un objet

On peut modifier un attribut après l'instanciation de l'objet.

```
# Modification d'un attribut  
voiture2.modele = "Accord"  
voiture2.afficher_details() # Marque : Honda, Modèle : Accord
```

Manipulation des attributs et des méthodes

Explication

1. `voiture2.modele = "Accord"` modifie directement l'attribut `modele`.
2. Lorsqu'on rappelle `afficher_details()`, la modification est prise en compte.

Résumé

Concept	Explication
Classe	Modèle définissant des attributs et des méthodes
Objet	Instance concrète d'une classe
Constructeur <code>__init__</code>	Méthode spéciale appelée lors de la création d'un objet
<code>self</code>	Référence à l'instance courante
Attributs	Variables propres à un objet
Méthodes	Fonctions définies dans une classe pour manipuler les objets

Conclusion

Les classes et les objets sont essentiels en POO pour structurer le code de manière modulaire et réutilisable. En comprenant l'instanciation, le rôle du constructeur `__init__`, et la manipulation des attributs et des méthodes, on peut concevoir des programmes plus organisés et maintenables.